Claims Orchestration on Temporal

Parent–Child Orchestration: a claim workflow with a child workflow per track

1. Summary

One parent workflow per claim, **ClaimLifecycleWorkflow** (Java), owns the claim's stage. Each visit to a track is a child workflow on that track's own task queue and worker: TriageWorkflow and InvestigationWorkflow (Python), StpWorkflow (Java). Each payment is a **PaymentWorkflow**, a child of STP that finishes even if STP is stopped. Change events go to the parent, which forwards them to the active child. Human decisions are validated Updates on the workflow that waits for them. Workflows carry IDs; ClaimsPro holds claim data; the projection table holds business history; both histories reach S3 before Temporal's retention deletes anything.

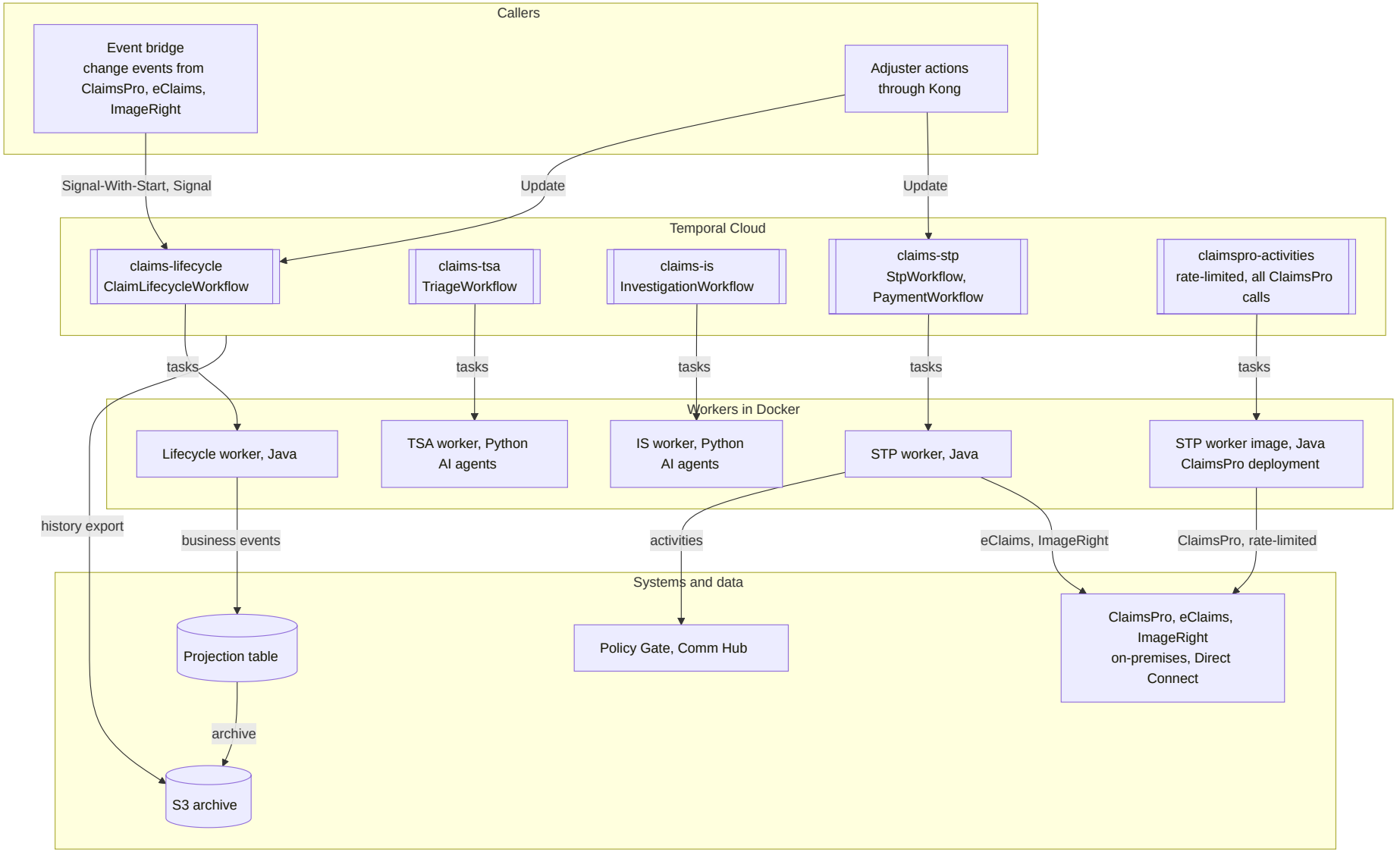
2. Why parent–child

Temporal uses child workflows when a unit of work has its own worker fleet, its own owner or its own lifecycle. Each child here has at least one of these.

Unit	Why it is a workflow of its own
Triage, investigation	AI workers; owned and released by the AI team
STP	Core workers; owned and released by the core team
Payment	Its own audit trail; must finish even if STP is stopped
Claim (parent)	The only place that changes the stage; holds routing and cross-track rules

Dispatch and appraisal have no separate owner or lifecycle, so they are activities inside STP.

3. Temporal architecture

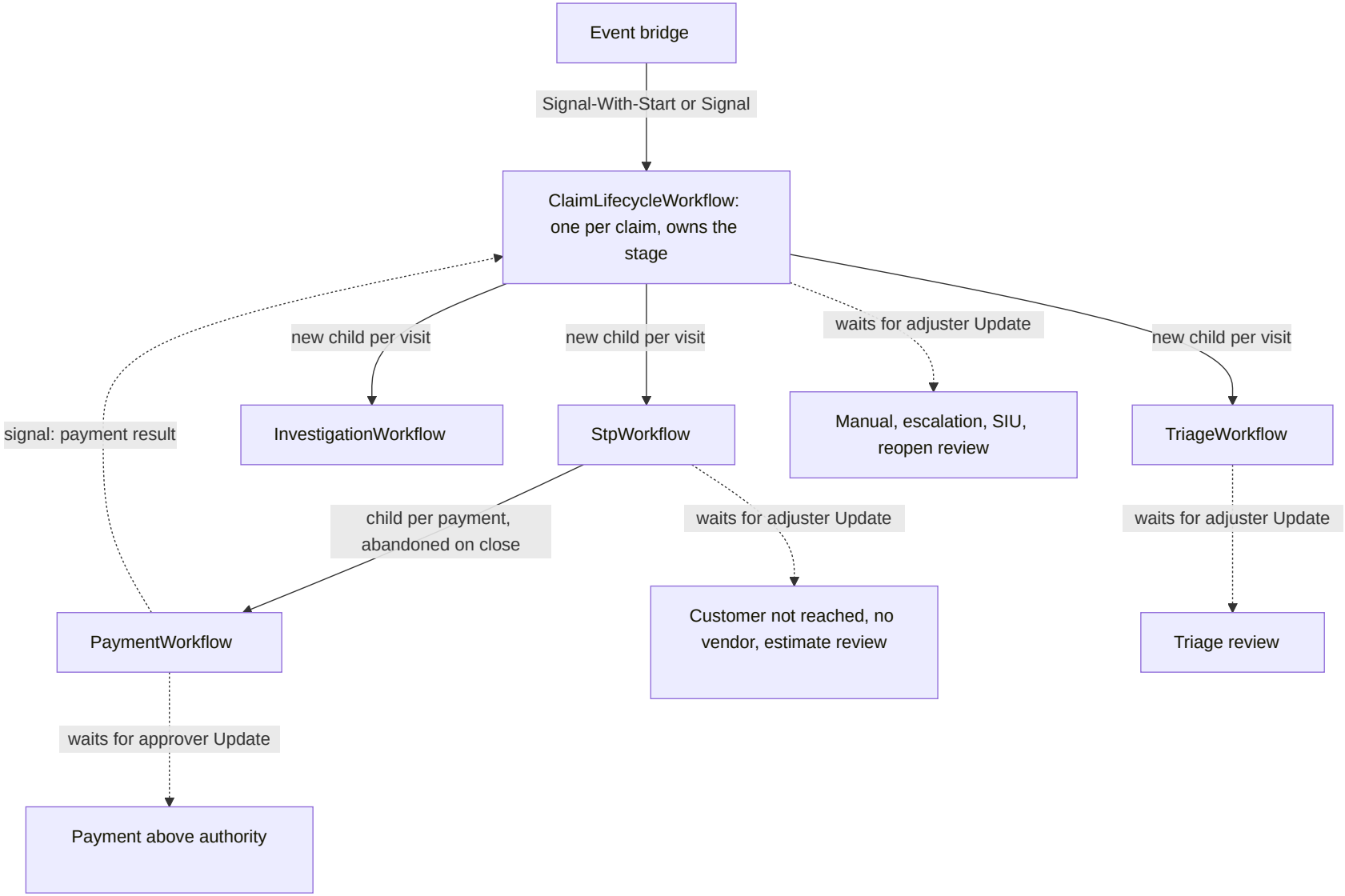


Task queue	Workflows and activities	Worker
claims-lifecycle	ClaimLifecycleWorkflow; projection writes; nightly archive and export check	Lifecycle worker, Java

Task queue	Workflows and activities	Worker
claims-tsa	TriageWorkflow; AI and triage rule activities	TSA worker, Python
claims-is	InvestigationWorkflow; AI coverage activities	IS worker, Python
claims-stp	StpWorkflow, PaymentWorkflow; Policy Gate, Comm Hub, eClaims and ImageRight activities	STP worker, Java
claimspro-activities	Every ClaimsPro call, from Java and Python workflows; rate limit set on the queue	STP worker image, run as its own deployment

One Temporal Cloud namespace per environment. Workers and the event bridge connect outbound only (gRPC with mTLS through PrivateLink). On-premises systems are reached over Direct Connect; inbound calls enter EKS through Kong. ClaimsPro calls run as their own deployment of the STP worker image, sized to ClaimsPro's limit. One EKS deployment per queue replaces today's single worker on EC2.

4. Workflows


Event bridge
Signal-With-Start or Signal
ClaimLifecycleWorkflow: one per claim, owns the stage
new child per visit
new child per visit
waits for adjuster Update
new child per visit
signal: payment result
InvestigationWorkflow
StpWorkflow
Manual, escalation, SIU, reopen review
TriageWorkflow
child per payment, abandoned on close
waits for adjuster Update
waits for adjuster Update
PaymentWorkflow
Customer not reached, no vendor, estimate review
Triage review
waits for approver Update
Payment above authority

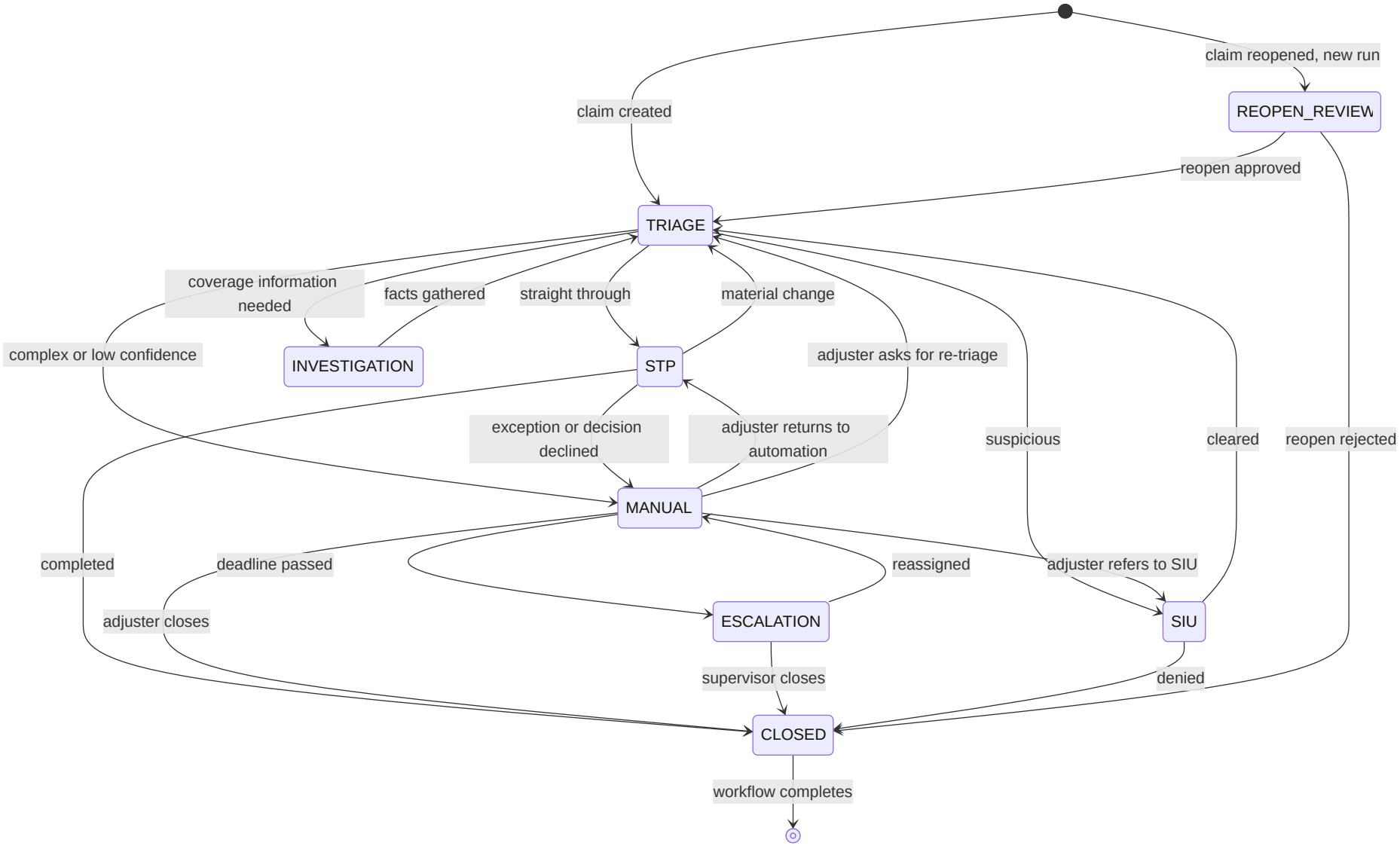
Workflow	Workflow ID	Lifetime	Messages	When its parent closes
ClaimLifecycleWorkflow	`claim:{claimId}`	Until the claim closes	Signals claimEvent, paymentResult · Update decide	—
TriageWorkflow	`claim:{claimId}:tsa:{stage}`	Minutes; longer with a review	Signal claimEvent · Update decide	Cancelled
InvestigationWorkflow	`claim:{claimId}:is:{stage}`	Hours to days	Signal claimEvent · Update decide	Cancelled
StpWorkflow	`claim:{claimId}:stp:{stage}`	Hours to days	Signal claimEvent · Update decide	Cancelled
PaymentWorkflow	`claim:{claimId}:payment:{paymentKey}`	Until closed, cancelled or rejected	Signal cancelPayment · Update decide	Abandoned: keeps running

A new child per visit keeps each history short. Its workflow ID carries the stage number, and the stage version on every write rejects a late result from an older stage.

The parent never has two children open. It stops a child by cancellation request, so the child withdraws its open tasks and writes its last row, and waits for it to close before starting the next. It continues as new only when no child is open. It completes when the claim closes; a later reopen starts a new run with the same workflow ID and continues the stage numbering.

PaymentWorkflow is abandoned, not cancelled, when STP closes, and reports its result to the parent by signal.

Claim stages



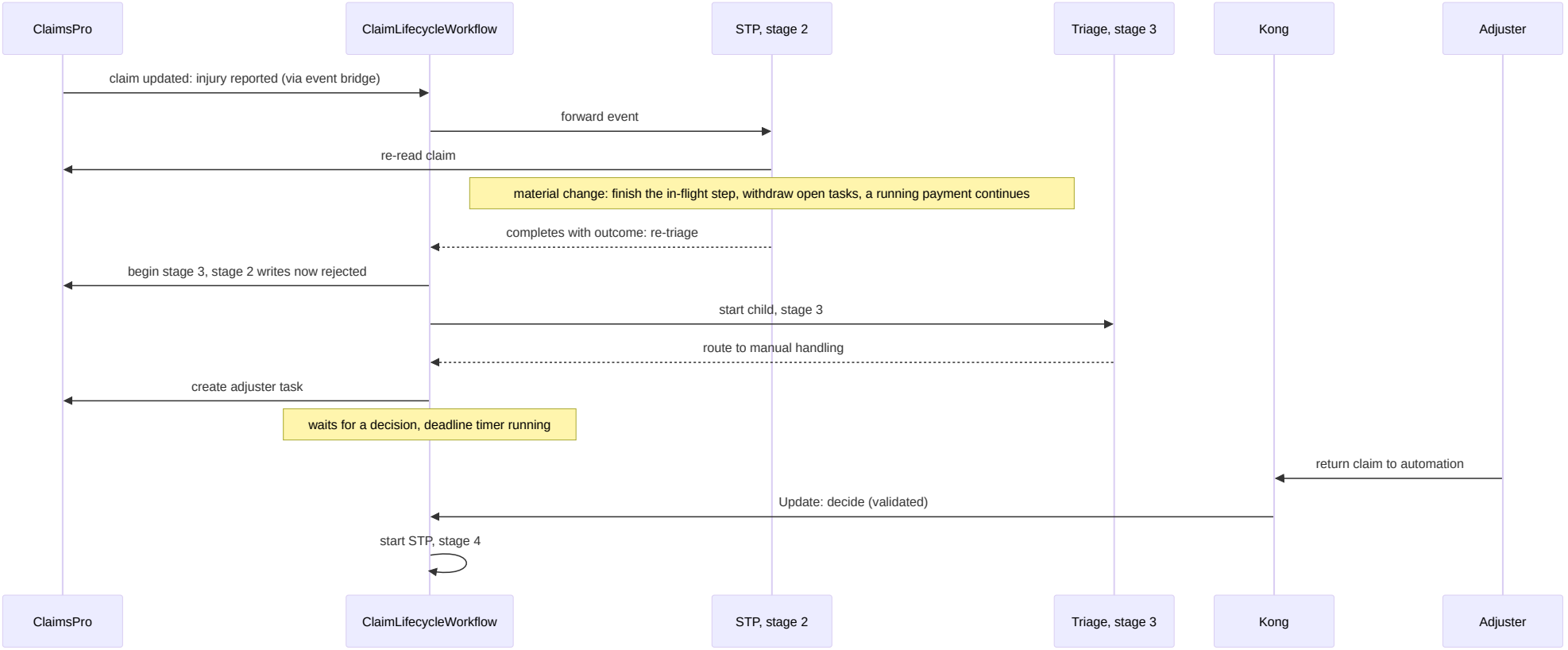
Every stage change increments the stage version in the projection table. Every writing activity checks it and refuses a write from an older stage; ClaimsPro needs no change. More re-triages than the configured limit (three by default) go to ESCALATION. A failed child or an unknown outcome goes to MANUAL.

STP steps

Step	Activities	System
1. Initial contact	Contact customer	Comm Hub
2. Coverage	Verify coverage	Policy Gate, ClaimsPro
3. Reserve	Open reserve	ClaimsPro
4. Estimate	Dispatch vendor and assign appraisal in parallel; wait for the estimate by event, with a timer that re-reads	eClaims
5. Payment	Start PaymentWorkflow for the balance ClaimsPro shows as unpaid; wait for its result	ClaimsPro
6. Close	Close claim	ClaimsPro

Steps and timers come from the STP flow definition for the claim's line of business, company, channel and state, produced from the variations tool. StpWorkflow records the definition's version and content hash at start and allows only registered steps. Payment authority and stage rules stay in code.

Re-triage

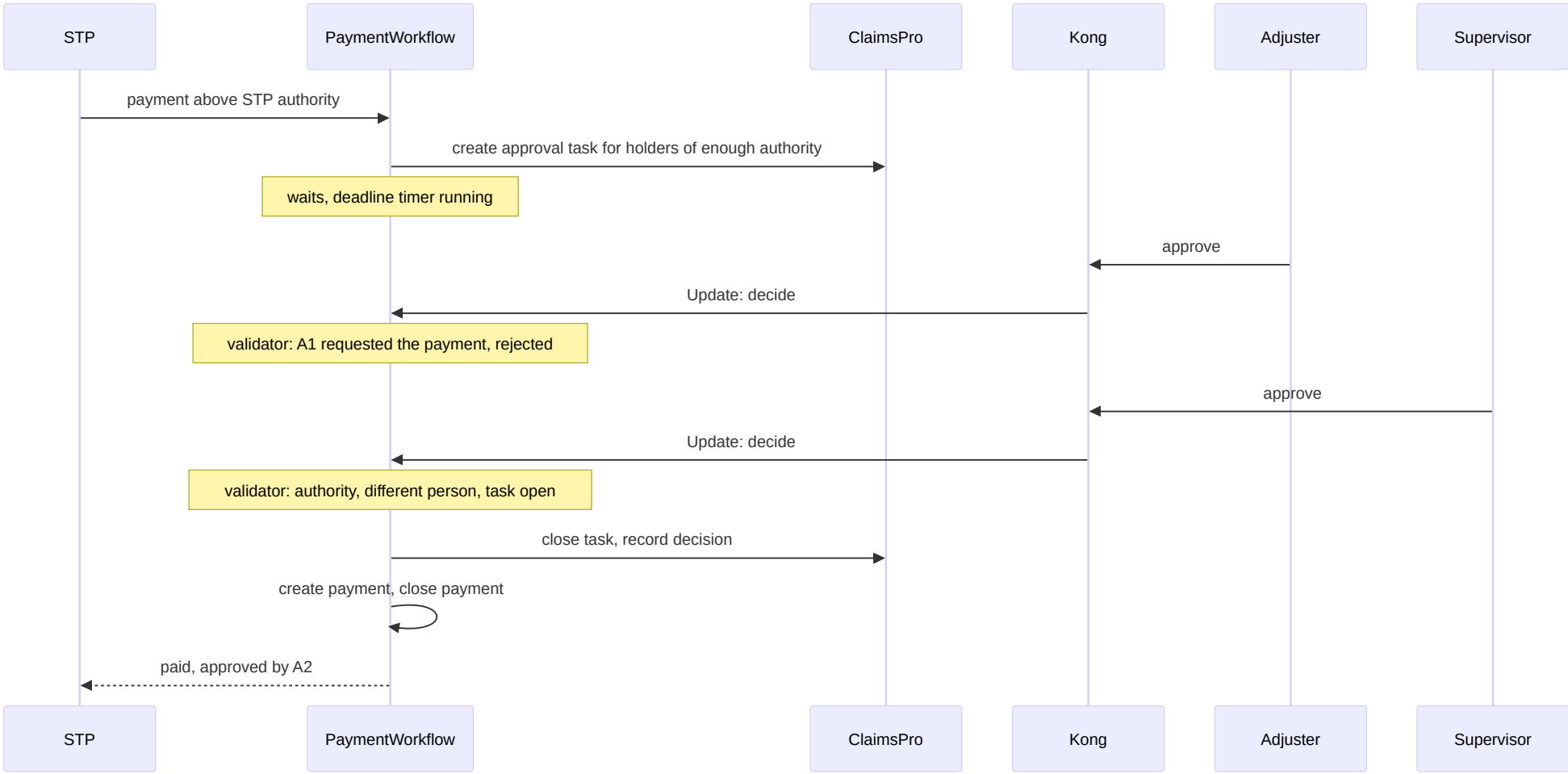


5. Human decisions

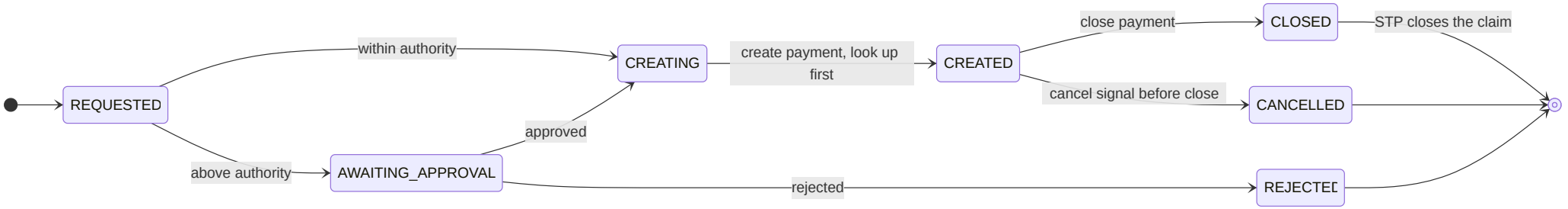
The workflow that needs a decision creates an adjuster task in ClaimsPro with its workflow ID and a decision ID, and waits with a deadline. The service behind Kong sends the adjuster's answer as Update `decide` to that workflow ID, with the decision ID as the Update ID. The validator rejects the decision before it is recorded if the task is closed, the choice is not offered, the role or authority is missing, the requester is the approver, or a required reason is missing (close, deny, reject). A repeated decision ID returns the first result. At the deadline the task moves to the next escalation role. Open tasks are withdrawn when the claim moves on.

Decision	Waits in	Decided by
Triage review when the AI result is below threshold	TriageWorkflow	Triage adjuster

Decision	Waits in	Decided by
Manual handling: close, return to STP, re-triage, refer to SIU	ClaimLifecycleWorkflow	Assigned adjuster
Escalation	ClaimLifecycleWorkflow	Supervisor
SIU: clear or deny	ClaimLifecycleWorkflow	SIU group
Additional coverage information	InvestigationWorkflow	IS reviewer
Customer not reached; no vendor accepts; estimate review	StpWorkflow	Adjuster
Payment above authority	PaymentWorkflow	Holder of enough authority, not the requester
Cancel payment	PaymentWorkflow (signal)	Adjuster
Reopen review	ClaimLifecycleWorkflow	Adjuster
A step still failing after its retry window: retry or hand to a person	The workflow that owns the step	Claims operations


STP
PaymentWorkflow
ClaimsPro
Kong
Adjuster
Supervisor
payment above STP authority
create approval task for holders of enough authority
waits, deadline timer running
approve
Update: decide
validator: A1 requested the payment, rejected
approve
Update: decide
validator: authority, different person, task open
close task, record decision
create payment, close payment
paid, approved by A2
STP
PaymentWorkflow
ClaimsPro
Kong
Adjuster
Supervisor

6. Payments



STP fixes the payment key when it requests the payment. Create looks up that key first, so a retry never pays twice. A cancel signal is honoured until the payment closes. The result goes to the parent and the projection table.

7. Activities and events

Activities	Timeouts and retries	Safe to repeat because
ClaimsPro: read claim, begin stage, reserve, payment, coverage, tasks, close claim	30 s per attempt; backoff 1 s to 60 s within a retry window; tasks and payment close retry until done	Stage version on every write; payment looks up before create
eClaims, ImageRight, Comm Hub, Policy Gate	30 s per attempt; retry window	Idempotency key `claimId:stageVersion:step`
AI agents (TSA, IS)	60 s per attempt; 5 attempts	Read only; a person decides below threshold
Projection write	10 s per attempt; retries without limit	Insert if absent on the same key

Not found, refused and stale stage are not retried. Other failures are retried until the step's retry window ends; then the workflow opens an operations task and waits, keeping the same idempotency key. Activities read current data from ClaimsPro by reference.

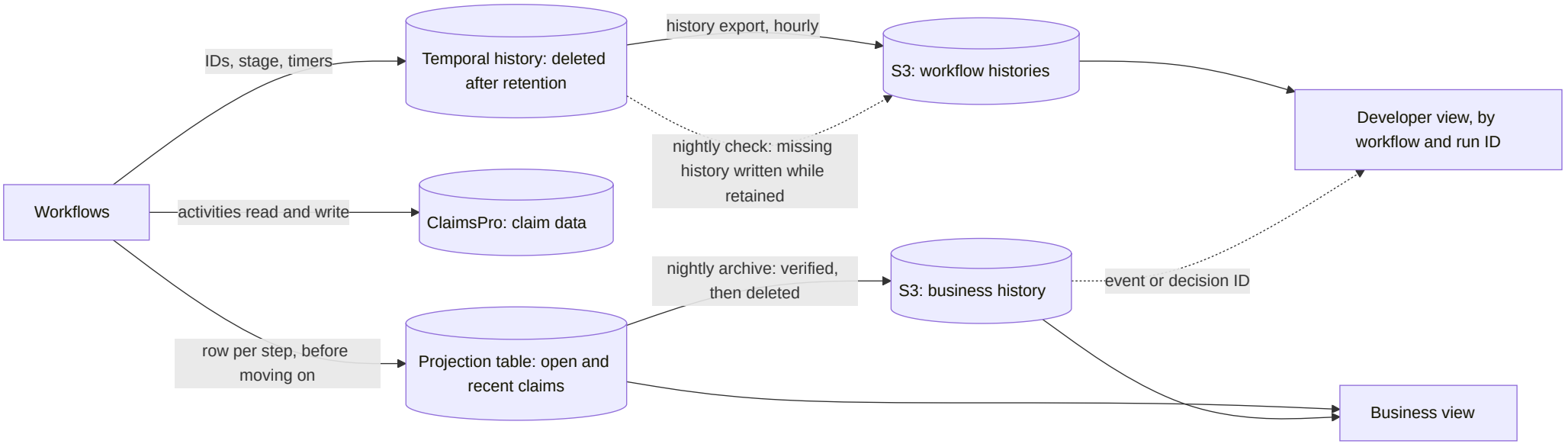
Event	Handling
Claim created or reopened	Signal-With-Start `claim:{claimId}`. The first activity reads the claim's status in ClaimsPro; a duplicate event for a closed claim ends the run at once

Event	Handling
Claim changed, estimate or documents received	Signal; deduplicated; forwarded to the active child, or held until one starts
Material change during STP	The in-flight step finishes; STP withdraws its tasks and completes with outcome re-triage; a running payment continues. Once money has moved, the change is recorded and STP closes the claim; a reopen brings it to an adjuster
Claim closed or reassigned directly in ClaimsPro	Active child cancelled; open tasks withdrawn; a payment not yet created is cancelled; stage moves to CLOSED or MANUAL
Payment cancelled in ClaimsPro	Forwarded by the claim workflow to the claim's payment workflow; honoured until the payment closes
Event for a claim with no running workflow	Recorded by the bridge for review; not started

The event bridge reads database change events over JDBC, keeps only allowlisted, non-personal fields, sends each claim's events in order, and advances its read position after Temporal accepts the signal. One signal per event, deduplicated by event ID; 100 to 150 events per claim is far below the 10,000-signal limit. Every wait for an external result also re-reads on a timer, so a lost event delays a claim but does not strand it.

8. Data, retention and S3

Temporal holds IDs, stage and timers. Everything needed later is written outside Temporal before Temporal can delete it: claim data in ClaimsPro, business history in the projection table, the archive of both in S3. Screens and reports read the projection table, not Temporal Queries.



Option for S3	Where it applies	Use here
Temporal Archival (S3 archiver)	Self-hosted Temporal only; experimental; not supported in Docker; looks runs up by exact workflow ID or type	Not available on Temporal Cloud
Temporal Cloud Workflow History Export	Temporal Cloud: every closed history to an S3 bucket hourly, as protobuf, at least once, within 24 hours; one action per workflow; set up with an IAM role and the client's KMS key	Used: the developer record
Projection archive	Our nightly workflow: business rows of claims whose every run has closed go to S3, verified, then exactly those rows deleted	Used: the business record

Kind of record	Where it lives	After retention
Workflow event history	Temporal Cloud	Exported to S3; Temporal Cloud has no Archival, export replaces it
Business history	Projection table	Archived to S3
Claim data	ClaimsPro	Unchanged; ClaimsPro is the system of record
Operational logs	ELK; LAMP for AI agents	Per the ELK retention policy

Kind of record	Where it lives	After retention
Temporal Cloud audit logs	Namespace and user administration only, no claim data	Per the audit log settings

Record	Contents	Written
Projection row: one per step, decision and event acted on	claimId, workflow ID, run ID, sequence number within the run, stage version, step, outcome, triggering event ID or decision ID, actor and reason for decisions, ClaimsPro references (reserve, payment), time	By an awaited activity, before the workflow moves on
Temporal history export	Full history: signals, Updates, activity inputs and results	Hourly by Temporal Cloud after the workflow closes, as protobuf
S3 business archive	Projection rows of claims closed more than N days ago, same schema, JSON lines partitioned by close date	Nightly by the archive schedule on the Lifecycle worker

Nothing is lost when retention ends

Risk	Guard
A step finishes but its projection row is missing	Insert-if-absent on `claimId:stageVersion:step`; the workflow does not move on until the write succeeds
A workflow closes with an incomplete record	Each workflow writes a closing row with its last sequence number; before ClaimLifecycleWorkflow completes, an activity confirms every run of the claim has contiguous rows up to its closing row
History export is late (up to 24 hours) or failing (disabled after 7 days)	Retention of at least 30 days. The nightly schedule confirms every workflow closed since its last run has an export in S3; a missing history is read from Temporal while it is still retained and written to S3
Projection rows deleted before the S3 copy is safe	Archive takes only claims whose every run has closed, verifies row count and checksum in S3, then deletes exactly the rows copied; a row written meanwhile waits for the next run; a rerun overwrites the same object
S3 objects deleted or changed	Bucket versioning and object lock for the regulatory period; encryption with the client's keys
Data that exists only in Temporal	None: payloads carry IDs, and every result that matters is written to ClaimsPro or the projection table

Data use cases

Use case	Served by
Business history of all claims, back in time, in one place	Projection table for open and recent claims, S3 business archive for older ones; same schema
An adjuster's 30 claims from four months ago, two still open	One query by adjuster over the projection table and the S3 archive together
A developer, six months later: why was this payment issued	The payment's projection row names the event or decision that triggered it; its workflow and run ID open the exported history in S3, showing the signal received and the activity run
Claims open for weeks or months	Fully in Temporal and the projection table; retention starts only at close

9. Contracts

Java and Python are generated from the same **Protocol Buffers** files (ClaimStart, ClaimEvent, TrackRequest, TrackResult, Decision, PaymentRequest, PaymentResult, ProjectionEvent and the activity messages). Payloads use protobuf JSON. Field numbers are never reused, fields are only added, enum value 0 is UNSPECIFIED and an unknown value goes to a person. No personal data appears in a message, workflow ID or search attribute.

10. Running in production

Area	Design
Catastrophe load	Namespace capacity sized from the catastrophe peak (500 actions per second by default). Workers scale per queue on schedule-to-start latency; the queue rate limit holds ClaimsPro at its agreed rate. Verified with the client's catastrophe load-test scripts.
Isolation	Separate task queues and worker deployments keep STP apart from TSA and IS load.
History size	Continue-As-New well below the 10,240-event warning; hard limit 51,200 events or 50 MB.
Billing (actions)	Workflow start 1; child workflow start 2; signal 1; Update 1, accepted or rejected; Query 1; search attribute upsert 1; activity start or retry 1; timer 1; history export 1 per closed workflow. Measured: about 67 actions for a straight-through claim, 50 of them activities.

Area	Design
Versioning	ClaimLifecycleWorkflow on Auto-Upgrade with patching, since it lives for the whole claim; tracks and payment Pinned, and each old pinned version kept until its last workflow closes. New versions ramped; recorded histories replayed in CI before release.
Recovery	A Temporal Cloud namespace is replicated across three availability zones. Multi-region replication is an option (recovery point under one minute, recovery time about 20 minutes); history export does not fail over with it. Choice depends on the recovery targets.
Security	mTLS and PrivateLink; one payload codec for Java and Python, failure messages included; codec server for authorised UI users. Search attributes and workflow IDs are not encrypted by the codec, so they hold IDs only. Keys rotate without making old histories unreadable; a worker with mTLS and no payload key refuses to start.
Testing	Time-skipping tests for races among signals, timers, activity completion, cancellation and Continue-As-New; replay tests on every release.
Observability	ELK logs carry claim, workflow and run IDs; LAMP for AI agents; Temporal Cloud and SDK metrics (Micrometer in Java) to dashboards. Search attributes ClaimId, Stage, Track, OpenDecision. Alerts: workflow task failures, queue latency, overdue decisions, steps past their retry window, export or archive failure.

11. Pros and cons

Pros	Cons and how they are handled
Standard Temporal pattern; each track owned and released by its own team	Every forwarded event costs one extra action (100 to 150 events per claim)
One owner of the claim stage; tracks cannot hand off to each other or loop without limit	A new child per visit adds actions and a short start-up delay at each stage change
Late or stale writes rejected by stage version; payments never paid twice or stopped halfway	Every writing activity reads the stage version first: one extra read per write
Human decisions answered at once and validated before they are recorded	Updates need a running worker; if workers are down the caller retries with the same decision ID
A catastrophe surge cannot overload ClaimsPro; tracks scale independently	The rate limit slows claims during a surge instead of failing them; peak duration must be acceptable to the business
No business or developer data is lost when retention ends	The projection table is on the critical path: if it is down, claims wait. Its availability must match Temporal's
Java and Python share one contract definition with enforced compatibility	Protobuf generation and the breaking-change check add a build step in both languages

Pros	Cons and how they are handled
STP variations change by configuration, not code	The flow interpreter must be versioned and replay-tested like workflow code

12. Answers to the questions in the design document and the kickoff call

Question	Answer in this approach
One parent coordinating TSA, IS, STP children? Where is the boundary?	Yes. The parent owns the stage, routing and cross-track rules only; each track owns its steps and returns an outcome. Children never call each other.
New child per track visit, or reuse one?	New child per visit; its workflow ID carries the stage number, so histories stay short and versioning stays per visit.
Stage versions and lease tokens to reject stale results?	Stage version on every write, checked against the projection table; a late write from an older stage is refused. No separate lease token.
Complete at close, or stay open for a reopen window?	Complete at close. A reopen starts a new run with the same workflow ID (Signal-With-Start) and continues the stage numbering. A reopen that arrives while the run is closing is handled in that run.
Workflow ID reuse, retention, Signal-With-Start issues	Only created and reopened events may start a run; a late created event for a closed claim ends at once. Retention starts at close; open claims are never deleted.
When to Continue-As-New	Between stages, with no child open, when Temporal suggests it or above 5,000 events (warning at 10,240).
Projection table for years? Wait for each write?	Yes, as the business record. Each row is written by an awaited, idempotent activity keyed claimId:stageVersion:step, so a step never moves on without its row.
Event history, Archival, audit logs, operational logs, projection: who keeps what?	Section 8: history in Temporal then S3 by export; business history in the projection table then S3; logs in ELK; audit logs for namespace administration only.
Recovery if a row is missing after history expires	Prevented: rows are awaited and the record is verified complete before the claim workflow completes; the nightly check copies any history missing from S3 while it is still retained.
Bridge to parent to child, or directly to children?	Bridge to parent; the parent forwards to the active child, holds events while no child runs, and every child re-reads ClaimsPro when it starts.

Question	Answer in this approach
100 to 150 events per claim: signal per event or coalesce?	Signal per event, deduplicated by event ID; far below the 10,000-signal limit. Each forward costs one action.
Human approval: Update or Signal? Parent or child?	Update with a validator, on the workflow that waits for the decision; rejected decisions never enter history; the decision ID is the Update ID.
JSON Schema or Protocol Buffers? Schema version? Compatibility?	Protocol Buffers, one definition for Java and Python. Fields only added, numbers never reused; unknown fields and enum values are read by both languages, and an unknown outcome goes to a person. Golden payloads are tested in both languages.
Condition change during an in-flight activity	The step in progress finishes; the stage version refuses any late write; a material change sends the claim to triage. A payment is never interrupted.
YAML/JSON flow definition	STP flows from the variations tool: registered steps only, version and content hash recorded per claim. Payment authority has a ceiling in code that configuration cannot raise.
Auto-Upgrade parent, Pinned children? How long do old versions run?	Yes. An old Pinned build runs until its last workflow closes. Every parent change passes replay of recorded histories.
Pass references or data to activities?	References. Activities read current data when they run, so a retry a day later uses the latest data.
A rerun must not create the same payment twice	Create looks up the payment key first. Verified: the reply to create is lost, the retry finds the payment, one payment is made. A later STP visit pays only the balance ClaimsPro shows as unpaid; once money has moved, STP closes the claim.
Create payment, close payment, close claim; cancel the payment	PaymentWorkflow creates and closes; a cancel from ClaimsPro reaches the payment through the claim workflow and is honoured until the payment closes.
Business view across open and archived claims; developer view months later	Projection table and S3 archive share one schema and are found by claim, adjuster or date; the payment's row names its trigger, and its workflow and run ID open the exported history.
Pilot scope: Claims Lifecycle to STP only	A policy switch sends claims straight to STP; a re-triage goes to an adjuster until TSA and IS are deployed.
Catastrophe load	Measured locally: 200 claims at 20 per second, all closed in 85 seconds with one payment each and complete records, with failures injected.
PrivateLink, encryption	Outbound only over PrivateLink with mTLS; payloads and failure messages encrypted with one codec in both languages. Keys rotate without making old histories unreadable; a worker with mTLS and no payload key refuses to start.

13. Claim scenarios verified end to end

Scenario	What it proves
Straight through	Full STP; a lost reply from ClaimsPro still gives exactly one payment
Injury during STP	Material change; re-triage; an adjuster returns the claim to STP
Coverage investigation	Investigation waits; documents arrive; re-triage; STP
Fraud	SIU denial without a reason is rejected; with a reason it is accepted
Large payment	Estimate review; approval rejects the requester and an approver without authority
Catastrophe, lost event	Catastrophe flow; the lost estimate event is recovered by the timer re-read
Customer not reached	An adjuster asks for another contact attempt
Low AI confidence	A triage adjuster chooses the route
Closed in ClaimsPro, reopened	The active track is cancelled and its task withdrawn; the reopen is a new run
Payment cancelled in ClaimsPro	A cancel during approval stops the payment; an adjuster closes the claim with a reason